

e-Borrow

Andrea Mirarchi

1 Cos'è il progetto

e-Borrowing è una web application ispirata a un marketplace come eBay, ma con un obiettivo diverso: lo scambio di oggetti in prestito invece della vendita.

Gli utenti possono:

- pubblicare richieste di oggetti in prestito;
- pubblicare offerte di prestito;
- prendere in carico richieste esistenti.

Il sistema include una gestione automatica della priorità delle richieste e un meccanismo di ordinamento delle offerte.

2 Casi d'uso

2.1 Caso d'uso 1 – Creazione richiesta

Un utente:

- inserisce una richiesta di prestito specificando oggetto, messaggio e tempo limite;
- pubblica la richiesta;
- rende la richiesta visibile agli altri utenti;
- consente agli altri utenti di prenderla in carico.

2.2 Caso d'uso 2 – Creazione offerta

Un utente:

- pubblica un oggetto disponibile al prestito;
- inserisce le informazioni relative all'offerta;
- rende l'offerta visibile agli altri utenti.

Le offerte vengono mostrate in ordine di rilevanza rispetto alla domanda degli utenti.

2.3 Caso d'uso 3 – Presa in carico

Un utente:

- seleziona una richiesta;
- decide di prenderla in carico;
- viene associato alla richiesta come prestatore;
- formalizza il prestito all'interno del sistema.

3 Come è implementato

3.1 Architettura backend

Il backend è sviluppato in Flask seguendo un'architettura a livelli composta da:

- `app.py`: gestione delle richieste HTTP e delle route;
- `services`: implementazione della business logic;
- `models`: definizione delle entità del dominio;
- `utils`: algoritmi di supporto, AI, ranking e matching;
- `repository`: gestione della persistenza dei dati.

3.2 Matching e ordinamento

L'associazione tra richieste e offerte è gestita tramite l'utility **matching**, che confronta gli oggetti richiesti con quelli offerti e individua le possibili corrispondenze.

Lo stesso meccanismo viene utilizzato per determinare la rilevanza delle offerte rispetto alle richieste presenti nel sistema.

3.3 Priorità delle richieste

La priorità delle richieste viene calcolata tramite la formula:

$$\text{Priorità} = \text{Urgenza} + \text{Importanza} - \text{Tempo}$$

I valori di urgenza e importanza vengono stimati automaticamente attraverso l'analisi del contenuto testuale della richiesta.

L'analisi è basata su un dataset di parole chiave e su un sistema di AI leggera che assegna un punteggio semantico al messaggio inserito dall'utente.

3.4 Gestione del prestito (Loan)

Quando una richiesta viene presa in carico viene generata un'entità **Loan**, che rappresenta il prestito attivo.

L'entità:

- collega il richiedente e il prestatore;
- rappresenta lo stato attivo del prestito;
- viene creata automaticamente dal service layer.

3.5 Frontend

Il frontend è realizzato utilizzando:

- Jinja2 come template engine;
- Web Components per la composizione dell'interfaccia;
- Bootstrap per la stilizzazione grafica.

3.6 Modello logico

Il modello è basato sul principio di ereditarietà.

Le entità principali sono:

- **BorrowEntity**: classe base comune;
- **BorrowRequest**: rappresenta una richiesta di prestito;
- **Offer**: rappresenta un'offerta di prestito;
- **Loan**: rappresenta un prestito attivo.

3.7 Persistenza dati

La persistenza è realizzata tramite file JSON Lines (`.jsonl`), scelti perché consentono scritture append-only più semplici e sicure rispetto a un unico file JSON tradizionale.

Per evitare problemi di concorrenza durante la scrittura viene utilizzato un meccanismo di lock.

3.8 Validazione dati

Il sistema implementa una validazione a due livelli:

- WTForms per la validazione dei form HTML;
- Marshmallow per la validazione dei dati ricevuti dalle API JSON.

3.9 Testing delle API

Per il collaudo delle API REST è possibile utilizzare **Flasgger**, estensione di Flask basata su Swagger.

Lo strumento permette di:

- documentare automaticamente gli endpoint;
- visualizzare parametri e formati dei dati;
- eseguire richieste HTTP direttamente dal browser;
- verificare le risposte restituite dal sistema.

L'interfaccia Swagger consente quindi di testare e validare rapidamente il comportamento delle API durante lo sviluppo, ed è raggiungibile tramite il path `/apidocs`

3.10 Routing

La navigazione dell'applicazione e la gestione delle API sono implementate tramite il decoratore `@app.route` fornito dal framework Flask.

3.11 Configurazione e variabili d'ambiente

La configurazione dell'applicazione è gestita tramite:

- `python-dotenv`;
- file `.env`;
- configurazioni centralizzate.

Le variabili d'ambiente vengono caricate all'avvio dell'applicazione e consentono di separare la configurazione dal codice sorgente.

Questo approccio permette di:

- modificare facilmente i parametri di esecuzione;
- gestire ambienti differenti (sviluppo, test e produzione);
- evitare l'inserimento diretto di dati sensibili nel codice.

3.12 Business logic

La logica applicativa è centralizzata nel service layer, responsabile delle operazioni principali del sistema.

In particolare:

- crea richieste e offerte;
- calcola le priorità;
- gestisce il matching tra utenti;
- genera i prestiti (**Loan**);
- coordina l'interazione con il repository dei dati.